

把知识图谱当代码库来逛 Schema-Agnostic Graph Reasoning Agent for Hybrid Knowledge Graphs

本篇范围说明：这是一篇公司白皮书形式的 arXiv 论文（2608.15834v1，2026-08-16，法国 Oplit 公司研发部，三位作者等贡献，标注日期 2026 年 7 月）。下面「全文翻译」一节按章节覆盖摘要、第一节的核心类比、三个对照系统的设计、基准的构造方式、四组结果与进一步分析、以及结论；第 8 节那两个工厂部署实例只取要点不逐段译，参考文献列表不译。

核心内容

Q：这篇论文的出发点是一个类比，那个类比是什么？

A：写代码的 agent 进到一个从没见过的代码库里也能干活，靠的只是几个通用原语：列目录、读文件、搜字符串。这些工具对项目本身一无所知——是 agent 通过导航把项目重建出来的。

而论文说：知识图谱接受同一套接口。

列出一个节点的邻居、读一个节点的内容、搜索节点的描述，是在另一个基底上结构等价的操作，所以写代码的 agent 那份导航能力应当以很低的代价迁移到知识图谱上。

这个类比的分量在于它换掉了一个默认假设。图谱问答这一行的默认做法是「先把问题翻译成一句图查询语言」；这篇的做法是不翻译，就一步步逛。

Q：「混合知识图谱」里的「混合」指什么？

A：论文假定的基底契约是最小的：每个节点带一个标识、一段自然语言描述、若干标签、以及可选的属性；关系是有向的主谓宾三元组。

而其中有些节点是「数据表」，背后是一张真实的 DuckDB 表、可以用 SQL 查。

这就是「混合知识图谱」里的「混合」：文本知识与关系型知识共存在同一张图里，而回答一个问题可能需要其中之一、或两者都要。

这个设计要解决的是一件很实际的事。有些问题的答案是一段定义（「什么叫延迟」——在语义层的概念节点里），有些是一个数（「上季度有多少订单延迟」——在数据层的表里），而大多数真实问题两者都要：先读定义搞清「延迟」的口径，再照这个口径去表里算。

Q: 那这篇要比较的两种哲学是什么？

A: 这是全篇的实验骨架。

两种哲学在争夺如何喂养这样一个基底。第一种把一切（文档与 schema）序列化进提示词，然后要一个答案。第二种给模型一个有界的上下文和一套工具，让它自己去取需要的东西。本文测量这个取舍，而且设计了对照来把结果归因到它的原因上。

「设计了对照来把结果归因到原因」这半句是这篇最讲究的地方，下面会看到它怎么做的。

Q: 三个被比较的系统分别是什么？

A:

系统	是什么	拿到什么
GRA (图推理 agent)	用七个 unix 风格工具 逛混合图谱	什么领域信息都没有——没有概念清单、没有标签字符串、没有表名，全部在运行时发现
RSA (检索式 SQL agent)	就是 GRA 把图去掉	在压平的文本文档分块与表 schema 上做检索
SQA (全上下文 SQL agent)	序列化一切那一派	提示词里就有完整的文本描述与完全渲染的 schema，约 1.7 万 token，最多 6 轮

GRA 的七个工具（论文刻意做成 unix 风格）：

工具	作用
ls	列节点、表、边，用来在图里定位方向
cat	读一个节点的全文；对表则显示列、键、连接关系、加 3 行样本
grep	在标识、文本、属性、边、列名上做字面搜索
sems	语义搜索（稠密向量 + BM25，取前 10）
query	只读 SQL（SELECT/WITH），结果上限 50 行
think	规划与自检的草稿纸，无副作用
answer	提交最终答案，可带引用与置信度

Q: RSA 这个对照组为什么重要？

A: 因为它把「agent 式访问」和「图结构」这两个变量分开了。

GRA 相对 SQA 有两处不同：一是访问方式（按需取 vs 全量塞），二是基底（图 vs 压平的文本）。如果只比 GRA 和 SQA，赢了也说不清是哪个原因赢的。

RSA 是「有 agent 式访问、但没有图」。而论文强调公平是由构造保证的：

两个 agent 跑的是**完全同一个执行循环**，它们的策略提示词块**逐字相同**，只在基底与工具集上不同。

这个对照设计是我在这篇里最看重的东西，因为它的结论恰恰否掉了论文标题暗示的卖点，见下。

Q：基准是怎么造的？

A：UFG-M（统一工厂知识模型），一个**虚构的自行车装配工厂**——受真实客户工厂启发但完全合成。有一份「立厂文本」用散文写出它的运营规则、关键指标、行业概念；然后两个协调的层把这个世界变成机器可读的：**数据层**（DuckDB 表）与**语义层**（知识图谱，它把立厂文本蒸馏出来并映射到那些表上）。

规模（论文主要用 xlarge 这一档）：**64 张表、17.4 万行、235 个图节点、513 条边、6933 词立厂文本、258 道题。**

而问题的生成方式值得单独说，这是这篇方法论上最结实的一处：**答案先于问题（answer-first）**。

问题是**倒着生成的：答案在问题存在之前就定了**。对每道题，一个大模型拿到一批 schema 卡片（表与列的描述）并写出一段 SQL 程序。这段程序**随后被真的在数据库上执行**。只有当结果非空、非退化、且不超过十行时才保留。只有到这时，大模型才写出一个能被那个保留结果回答的自然语言问题。

为什么这个顺序重要，论文自己说了：

因为 SQL 是在问题之前写好并验证过的，所以这个集合里的每道题都是能从数据里答出来的，而它的**标准答案是一段真的跑过的程序的输出，而不是模型产出的文本。**

配套的评分也不用大模型当判官：由一个确定性的匹配器判对错——数值答案按舍入精度容差比较，百分比答案与**尺度无关**地比较（不管写成分数还是百分数），表格答案按它必须包含的标准行的召回率打分。

258 道题的结构：116 道答案是表格、84 道是单个值、48 道是布尔、10 道是列表；147 道最多一个连接、45 道两个、**66 道三个或更多**；另有 **34 道额外需要语义层**（一条命名的运营规则或指标，其解析后的值或公式是藏起来的）。

Q：结果如何？

A：**头条数字是 GRA 88.4% 对 SQA 83.3%，高 5.1 个百分点，而只读了不到三分之一的输入 token。**

但真正的信息在拆开之后。

第一层拆开：谁赢取决于模型。

「agent 式方法与全上下文方法的排名取决于模型」——GRA 在 DeepSeek 与 GLM 上最好，**而 SQA 在 Qwen3-Coder-Flash 上更好（比 GRA 高 2.3 点）、在 GPT-5 Nano 上更好（高 5.5 点）。**

第二层拆开，这是全篇最有用的一条：工具调用的可靠性比延长推理更要紧。

模型（跑 GRA）	准确率	平均工具调用次数	失败调用占比	至少一次失败的题占比
DeepSeek V4-Pro-Think	88.4%	12.1	0.5%	5.4%
DeepSeek V4-Flash	87.6%	14.8	0.7%	8.1%
DeepSeek V4-Pro	87.2%	13.4	0.6%	7.0%
Qwen3-Coder-Flash	78.7%	12.2	2.0%	15.9%
GLM-4.5-Air	77.5%	11.1	2.4%	18.2%
GPT-5 Nano（开推理）	76.7%	11.6	5.8%	34.9%
GPT-5 Nano	70.5%	11.1	10.2%	51.6%

读法：准确率的排序和「失败调用占比」的排序几乎完全一致。三个 DeepSeek 配置失败率都在 1% 以下、占据前三；GPT-5 Nano 失败 10.2%、在超过一半的题上至少漏一次调用，而它正是唯一 SQA 反超 GRA 的那个模型。

而「延长推理」几乎没用：三个 DeepSeek 配置彼此相差 1.2 个点以内、区间重叠。唯一的例外是 GPT-5 Nano 涨了 6.2 点，但它的失败率同时大约减半（10.2% → 5.8%）——所以论文判断这个增益是由可靠性中介的，不是由推理深度带来的。

这些结果表明，在这个任务上，可靠的工具使用是比延长推理更强的瓶颈。

第三层拆开：token 用量的方向是反的。

输入：GRA 读 SQA 的 29 - 33%，RSA 读 24 - 29%。输出反过来：SQA 每题只产出 0.5 - 1.3k 完成 token（因为它用一个 SQL 块作答），而多轮的 agent 产出 1.0 - 2.0k。

而这里论文做了一个很诚实的成本区分，值得记：

唯一 token 与计费 token 是两回事。GRA 与 RSA 在 11 到 15 轮里反复重发一个不断增长的上下文，而 SQA 平均不到三轮。在缓存感知的定价下，相对成本取决于负载形态：热缓存的批量评测偏向 SQA，因为它那个巨大的固定前缀可以跨问题复用；冷启动的单问题服务偏向 agent，因为 SQA 每道题都要付它那 1.7 万 token 的提示词。

Q：工具调用预算给多少合适？

A：预算从 10 加到 50，准确率在 10 到 30 之间陡升、之后走平。同一区间里被截断（预算耗尽却还没主动作答）的题数从 118 掉到 11（xlarge 上）。预算低于 20 经常截断搜索、拉低准确率——xlarge 上 B=10 时准确率只有 63.6%。30 以上观察不到可测量的增益。

所以「大约 30 次调用」是这个数据集上的拐点。而一旦截断变得罕见，平均轮数就稳定在 13 轮左右（xlarge）——也就是说给到 30 的预算，实际只用 13 轮。预算的作用不是让它多走，是让它别在半路被掐

断。

Q: 那图结构本身贡献了多少?

A: 这是论文最诚实的一段, 而它的答案是「很小」。

图的拓扑结构的贡献则要不清楚得多: GRA 只比 RSA 高 0.3 到 1.9 个点, 而在 DeepSeek V4-Pro 上两者不可区分, 这表明相对 SQA 的增益主要来自上下文被如何访问, 而不是来自图结构本身。

换句话说: 赢的是「按需取」, 不是「有图」。摘要里就把这一点写出来了——「一个无图的对照显示增益主要来自选择性的 agent 式访问, 而不是图的拓扑」。

Q: 论文自己划的边界?

A: 两条, 都在「进一步分析」那一节。

一, 可靠性划定了这个行为的边界: 一旦工具调用失败率超过百分之几, agent 式外壳的优势就缩小或反转——这就是为什么对较弱的工具调用者, 全上下文推理仍然更可取。

二, 最该发挥作用的那个场景恰恰没被测到:

当前的语料小到 SQA 那 1.7 万 token 的提示词能舒舒服服装进每个模型的上下文窗口, 所以结构化导航本应最有帮助的那个区间——序列化不可行或代价过高的时候——仍有待在大得多的图上检验。

背景补充 / 点评

先说这篇和 prove 的关系, 因为它几乎是在同一个问题上做的对照实验。

prove 要做的事和 UFK-M 上的问答是同一类: 用户用自然语言问一个业务问题, 系统要搞清口径、找到表、算出数、给出答案。prove 现在的形态是固定流水线: 路由(选指标) → 编译 → 执行 → 派生计算 → 对账 → 作答。

按前几天那篇 Agentic SQL 的自主度分档, prove 是 L2 (多次调用沿固定流水线, 无动态路由)。而 GRA 是 L3 (有控制器在中间产出上分支)。所以这篇给的是「往 L3 走会得到什么、要付什么」的一份实测。

它给的三条数据都指向同一个判断: 那笔账不划算, 至少现在不划算。

第一条, 赢的不是图也不是 agent 结构, 是「按需取」。GRA 比 RSA 只高 0.3 到 1.9 点、在最强的模型上不可区分。而 prove 本来就是按需取的——它不把整个数仓 schema 塞进提示词, 路由那一步就是在选要看哪一部分。也就是说这篇测出来的那个主要增益, prove 已经拿到了。

第二条, 瓶颈在工具调用可靠性, 不在推理能力。那张表里准确率排序和失败率排序几乎一致, 而延长推理在可靠的模型上只值 1.2 点以内。这一条对 prove 的直接含义是: 如果端到端准确率不够, 先去数「工具调

用/编译/执行环节失败了多少次」，再考虑换更强的模型或加推理预算。这是一个可以立刻做的测量，而且它的成本几乎为零——日志里应该已经有这个数了，只是没有按「失败调用占比」和「至少一次失败的题占比」这两个口径统计过。

这两个口径要分开，因为它们说的不是一回事：前者是「调用总数里坏了多少」，后者是「多少道题至少被坏影响过一次」。GPT-5 Nano 那行 10.2% 对 51.6% 就是极端例子——每十次调用坏一次，就足以污染一半的题。这个放大倍数是多轮 agent 的固有代价。

第三条，多轮 agent 的成本形状取决于负载。论文那段区分很实在：热缓存的批量跑偏向大提示词单次调用，冷启动的单问题服务偏向多轮 agent。prove 的评测集是批量跑（47 道题一轮），而生产是单问题服务——这意味着评测阶段量到的成本，和生产里的成本形状是不一样的，不能直接外推。这一条我此前没意识到。

再说这篇方法论上最值得抄的一处：答案先于问题。

它的做法是先写 SQL、先跑出结果、只有结果非空非退化才保留、然后才让模型写一个能被这个结果回答的问题。

这解决的是评测集构造里一个很难的问题：怎么保证每道题都真的有答案，而且标准答案不是模型编的。常规做法是先写问题再标答案，于是有两个风险——问题可能根本答不出来（数据里没有），标准答案可能是模型幻觉出来的。倒着来两个风险一起消掉。

prove 那 47 道题的评测集是正着造的（先有问题，再定期望答案）。按这篇的判据，那个集合里可能混着两类噪声：答不出来的题（会被记成模型失败，其实是数据缺失），和标准答案本身有问题的题。这不是要现在推翻重造，但它给了一个具体的排查方向：把那些长期答不对的题拿出来，先验证「这题的标准答案是否由一段真跑过的查询产出」。

配套还有一条：评分用确定性的匹配器而不是大模型判官，并且百分比答案与尺度无关地比较（0.35 和 35% 算同一个答案）。这一条细节很小但很有用——这类格式差异造成的假失败，在指标类问答里极其常见。

第三处值得记的是那个「拐点在 30 次调用、实际只用 13 轮」的结构。

它说明预算的作用不是鼓励多走，而是给足余量、别让它在半路被掐断。B=10 时 xlarge 上有 118 道题被截断、准确率只有 63.6%；B=30 时截断掉到 11 道。而 B=30 与 B=50 没有可测量差别——因为平均只用 13 轮，多给的预算根本没被用到。

这个形状的一般版本是：一个有上限的迭代过程，性能对上限的敏感区间只在「上限接近实际需求」的那一段；上限一旦明显高于实际需求，再加就是零收益。所以定这类上限的正确方法不是猜，是先量出实际用量的分布，再把上限设在分布的尾部之外。

顺带对照一下今天读的另一篇（SABET-QA），它测的是多跳深度 K 从 1 到 8，发现时间类问题在 K=8 时比 K=4 差了 7.9 个点——也就是更深反而更糟。两篇的结论不冲突，因为管的是两件事：GRA 那个 30 是「允许调用的上限」（走不完会被掐断，给足就行，多给无害）；SABET-QA 那个 K 是「强制展开的层数」（每一层都要算，多了会累积噪声）。上限和固定层数是不同的东西，混淆了会得出相反的结论。

最后说这篇该保持怀疑的地方。

第一，基准是自己造的、完全合成的，而作者是要卖这套东西的公司。 UFK-M 是虚构工厂，「受真实客户工厂启发但完全合成」。合成有好处（能保证每题有答案、能控制规模），但它也意味着语义层与数据层之间的映射是设计者亲手对齐好的——真实企业里那份对齐通常是缺失的、过时的、或者互相矛盾的，而恰恰是那种情况下「读一条边就知道概念挂在哪张表上」这个便利会消失。论文没有测这个。

第二，那个 5.1 点的头条数字只在特定模型上成立。 GRA 在两个模型上是输的（Qwen3-Coder-Flash 输 2.3 点、GPT-5 Nano 输 5.5 点）。摘要写的是「beats a full-context agent by 5.1 pp」，正文写的是「排名取决于模型」——这两句的分量差很多，而只有后者是完整的。论文在正文里说清了，这一点算诚实，但读摘要会被误导。

第三，最该起作用的场景没测，论文自己承认了。 语料小到 1.7 万 token 能装进任何模型的窗口——也就是说这个实验测的是「在不需要导航的规模上，导航是否也不亏」，而不是「在必须导航的规模上，导航有多好」。后者才是这套方法的立论基础，而它是空的。

第四，图结构贡献不清这一条，比论文写得还要更弱一点。 GRA 高 RSA 0.3 到 1.9 点，而 258 道题的规模下，1 个点大约是 2.6 道题。在最强的模型上两者「不可区分」，那么标题里的「Graph Reasoning」到底贡献了什么，这篇答不了。它测出来的其实是「一个 schema 无关的 agent 用通用工具按需取上下文，比把一切塞进提示词更好（在能可靠调工具的模型上）」——这个结论很有价值，但它和图没什么关系。

一句话版本：这篇的价值不在它标题里的「图」（无图对照组只落后 0.3 到 1.9 点，最强模型上不可区分），而在三件副产品——工具调用可靠性是比推理深度更硬的瓶颈（失败率排序几乎就是准确率排序，且 10% 的调用失败率能污染一半的题）；「答案先于问题」的评测集构造法一次消掉两类噪声；以及多轮 agent 与大提示词的成本孰优取决于是热缓存批量还是冷启动单问。

文中金句

"Tool-calling agents work effectively inside repositories they have never seen, using a small set of generic primitives: list a directory, read a file, search for a string. None of these tools knows anything about the project; the agent reconstructs it by navigating." 会调工具的 agent 在**从没见过代码库里也能有效工作**，用的只是一小套通用原语：列目录、读文件、搜字符串。这些工具对项目一无所知；是 agent 通过导航把它重建出来的。

"Listing the neighbours of a node, reading a node's content, and searching node descriptions are structurally equivalent operations on a different substrate." 列出一个节点的邻居、读一个节点的内容、搜索节点的描述，**是在另一个基底上结构等价的操作。**

"textual and relational knowledge coexist in one graph, and answering a question may require either or both." 文本知识与关系型知识共存在同一张图里，而回答一个问题可能需要其中之一、或两者都要。

"Two philosophies compete to feed such a substrate. The first serializes everything into the prompt and asks for an answer. The second gives the model a bounded context and a toolset, and lets it fetch what it needs. This paper measures that trade-off, with controls designed to attribute the outcome to its cause." 两种哲学在争夺如何喂养这样一个基底。第一种把一切序列化进提示词然后要一个答案。第二种给模型一个有界的上下文和一套工具，让它自己去取。本文测量这个取舍，而且设计了对照来把结果归因到它的原因上。

"Fairness is enforced by construction: both agents run the very same execution loop, share their strategy prompt blocks verbatim, and differ only in substrate and toolset." 公平是由构造保证的：两个 agent 跑完全同一个执行循环、策略提示词块逐字相同，只在基底与工具集上不同。

"Questions are generated in reverse: the answer is fixed before the question exists." 问题是倒着生成的：答案在问题存在之前就定了。

"Because the SQL is written and validated before the question, every question in the set is answerable from the data, and its gold answer is the output of a program that has actually run rather than text produced by a model." 因为 SQL 是在问题之前写好并验证过的，所以集合里每道题都能从数据里答出来，而它的标准答案是一段真跑过的程序的输出，而不是模型产出的文本。

"These results indicate that reliable tool use is a stronger bottleneck than extended reasoning on this task." 这些结果表明，在这个任务上可靠的工具使用是比延长推理更强的瓶颈。

"the gain over SQA comes mainly from how context is accessed rather than from graph structure itself." 相对全上下文基线的增益主要来自上下文被如何访问，而不是来自图结构本身。

"the advantage of the agentic harness shrinks or reverses once tool-call failures exceed a few percent, which is why full-context inference remains preferable for weaker tool-callers." 一旦工具调用失败超过百分之几，agent 式外壳的优势就缩小或反转——这就是为什么对较弱的工具调用者，全上下文推理仍然更可取。

"the regime in which structured navigation should help most, where serialization is infeasible or too costly, remains to be tested on substantially larger graphs." **结构化导航本应最有帮助的那个区间——序列化不可行或代价过高的时候——仍有待在大得多的图上检验。**

"Seeing less, the agent answers better: selective navigation over a structured substrate beats exhaustive context." **看得更少，答得更好：** 在一个结构化基底上做选择性导航，胜过穷尽式的上下文。

全文翻译

摘要

会调工具的大模型 agent 只用少数几个通用原语（`ls`、`cat`、`grep`）就能在陌生代码库里导航。**知识图谱接受同一套接口：**列邻居、读节点内容、搜描述，是在不同基底上的同一批操作。

Building on this correspondence, we present GRA, a Graph Reasoning Agent that explores hybrid knowledge graphs, whose nodes are either textual concepts or relational tables, with seven generic tools, discovering everything domain-specific at run time.

基于这个对应，我们提出 **GRA**——一个图推理 agent，它用**七个通用工具**探索混合知识图谱（其节点或是文本概念、或是关系型表），**一切领域相关的东西都在运行时发现。**

On UFK-M, an industrial benchmark of 258 analytical questions whose gold answers are produced by executing validated SQL programs, GRA beats a full-context agent by 5.1 pp (88.4% vs. 83.3%), while reading under a third of its input tokens.

在 UFK-M 上——一个 258 道分析性问题的工业基准，**其标准答案由执行经过验证的 SQL 程序产出——**GRA 比一个全上下文 agent 高 5.1 个百分点（88.4% 对 83.3%），**同时只读了它不到三分之一的输入 token。**

A graph-free control shows the gain comes chiefly from selective agentic access rather than graph topology, and that the effect depends on a model able to drive tools reliably.

一个无图的对照显示：这个增益主要来自选择性的 agent 式访问、而不是图的拓扑，而且这个效应取决于模型能否可靠地驱动工具。

Seeing less, the agent answers better: selective navigation over a structured substrate beats exhaustive context.

看得更少，答得更好：在结构化基底上的选择性导航胜过穷尽式上下文。

一、从代码 agent 到图 agent

（核心类比与基底契约见「核心内容」一节，此处不重复。）

论文强调基底契约是**最小的**——节点有标识、自然语言描述、标签、可选属性；关系是有向三元组；部分节点是背后有真实 DuckDB 表的数据表。这就是「混合」的含义。

二、相关工作

GRA 的接口传承自 **ReAct**，它引入了推理与工具调用交错这个现在很普遍的做法，**但没有回答一个给定基底应该暴露哪些工具**。对源代码，**SWE-agent** 回答了这个问题：它的「agent—计算机接口」表明少数几个文件系统命令就足以在陌生仓库上工作。

在这一点上，知识图谱很像一个代码库——**一个通过跟随链接来探索、而不是整本读完的大型连通结构**——所以那套策略可以直接迁移，**只是基底不同**。

另一条平行的路线给语言模型直接提供结构化知识的接口（图检索增强生成那一系）。这些系统在图、表、数据库上读取，跟随关系路径，或暴露节点查找与邻居列举的原语。

All share GRA's premise — navigate the graph rather than read a flat dump of it — but assume that the graph's vocabulary is known and that traversal alone answers the question. Neither assumption holds on a hybrid substrate, where the deciding fact often sits in a table reached through the graph, and where a question's words must first be matched to a node.

它们都共享 GRA 的前提——**导航图谱而不是读它的压平转储**——但假定图谱的词汇是已知的，且遍历本身就能回答问题。在混合基底上这两个假定都不成立：决定性的事实常常在一张要通过图才能到达的表里，而问题的用词必须先被匹配到某个节点上；把这两者对齐本身就很难。

GraphRAG 则是**离线**建图并做摘要、再在其上检索，这适合宽泛的问题，但它把检索结构预先固定了；而 GRA 是按每个问题、在一张持续变化的图上采集证据。

论文对自己定位的说法是：这些工作各自提供了 GRA 需要的一部分，**但没有一个同时提供全部三样**——不做逐 schema 调优就能运行、一个把语义图与关系表联合起来的基底、以及计算一个没有任何节点存储的量的能力。

三、三个 agent，一个基底

（三个系统的设计与七个工具见「核心内容」。）

论文给了同一道题（「上季度每个客户细分有多少订单延迟」）在三个系统下的完整调用轨迹：

GRA: `ls` (标签) → `ls` (表) → `grep("orders")` → `grep("customer segment")` → `cat` (订单表) → `cat` (客户概念) → `cat` (客户表) → `query` → `think` → `answer`。agent 是冷启动的：没有

schema、没有表名、没有词汇。图谱与「延迟」的定义都是在路上发现的，而全程只读了几千个唯一 token。

RSA：同一个循环走到同一个答案，但「客户细分」这个概念与存它的那张表之间的联系，必须从检索到的文本分块里推断出来、而不是从一条边上直接读出，候选表是按名字扫描而不是被跟随的。

SQA：提示词里已经有立厂文本与每张表的 schema（约 1.7 万 token）。第一轮：一句连接两张表的 SQL。第二轮：答案。

The whole corpus is read before the first word, whether or not the question needs it, and the remaining turns serve only to repair a failed query.

整个语料在第一个词之前就被读完了，不管这道题需不需要，而剩下的轮次只用来修一个失败的查询。

四、UFK-M 基准

（构造方式、规模、答案先于问题的生成法、确定性评分见「核心内容」。）

五、实验设置

评测七个底座配置、跨四个供应商：DeepSeek V4-Flash（非思考，作为参照）、DeepSeek V4-Pro 与 V4-Pro-Think（**同一份权重，推理关与开**）、GPT-5 Nano（低/高推理强度）、GLM-4.5-Air、Qwen3-Coder-Flash。

每个底座跑三种基线，**GRA/RSA 给 45 轮大模型调用、SQA 给 6 轮**；SQL 工具结果上限 50 行；GRA/RSA 用一个本地嵌入模型检索，SQA 则在提示词里直接拿到完整 schema、不做检索。**所有模型贪心解码（温度 0），完成长度上限 1024 token（思考配置 8192）以避免答案被截断；其余装置在各系统间完全相同。**

不确定性用**按问题的配对自助法**量化（一万次重采样），报 95% 分位区间。

六、结果

（准确率、工具可靠性、token 用量、预算拐点四小节的数据见「核心内容」。）

关于跨模型的比较，论文的原话是：DeepSeek 系列在所有系统下都取得最高准确率，**在 GRA 与 RSA 下领先 8 到 18 个点，而在 SQA 下只领先 2 到 9 个点**。低成本模型中，Qwen3-Coder-Flash 在每个系统上都显著优于 GPT-5 Nano。

这个「**在 agent 下差距被放大、在全上下文下差距被压缩**」的现象本身有含义：agent 式方法对模型能力更敏感，因为它把成败押在多轮工具调用上。

七、进一步分析

Taken together, the results identify selective agentic access as the primary source of GRA's advantage on this benchmark.

综合起来，这些结果把选择性的 agent 式访问认定为 GRA 在这个基准上优势的主要来源。

在能可靠调用工具的模型上，两个 agent 都优于全上下文基线，而 GRA 取得最大的、有良好支撑的增益：**+5.1 个点，同时只读 SQA 唯一输入 token 的 29% 到 33%。**

This efficiency is expected to matter more as the corpus grows beyond what a single prompt can hold, since SQA's cost scales with the full serialized context whereas the agents read only the fraction they retrieve.

预期这个效率会随着语料超出单个提示词所能容纳的范围而更加要紧，因为全上下文基线的成本随完整序列化上下文增长，而 agent 只读它检索到的那一小部分。

图拓扑的贡献则**不清楚得多**：GRA 只比 RSA 高 0.3 到 1.9 点，在 DeepSeek V4-Pro 上两者不可区分。

可靠性划定了这个行为的边界：一旦工具调用失败超过百分之几，agent 式外壳的优势就缩小或反转。

最后，论文自己指出最该起作用的区间还没测（原文见「文中金句」）。

八、部署闭环（要点）

前面几节把 GRA 当作一个问答模块来测量。在工厂里，同一个模块坐在一个更大的闭环中：**操作员用平白语言说出一条规则；一个编排器接到它，问 GRA 这条规则是否可行；GRA 通过用那些通用工具导航图谱来采集证据，并返回一个带引用的裁决。**

部署时**多加一个写原语 edit**，只用来登记已批准的规则；它不在被评测的工具集里。被接受的规则交给 ORA（运筹 agent），它把规则变成数学优化模型与求解器代码，**并把它作为新的规则节点登记回图里。**

两个实例分别对应两种结局：**一条必须被拒绝的规则**（论文的例子是操作员说「铝制车架周一去焊接工位 1 或 2，工位 3 在检修」——这句话清晰、格式良好、能编译成一条合法约束，但它是不可可能的，而不可可能的两个理由分别住在两个不同的地方），以及一条被接受并编译的规则。

这个例子的意思是：判定可行性需要同时用到语义层的规则和数据层的数字，而这正是「混合」基底与「计算一个没有节点存储的量」这两项能力的联合用途。

九、结论

（结论重申摘要的三条：类比成立、增益来自选择性访问而非图拓扑、效应依赖模型的工具调用可靠性。）

参考链接

- [论文 arXiv 2608.15834](#) — Oplit 研发部白皮书，2026-08-16

- [ReAct](#) — 推理与工具调用交错的起点，GRA 的接口传承自它
- [SWE-agent](#) — 「几个文件系统命令就够用」这个答案的出处，本文的核心类比来源
- [GraphRAG](#) — 离线建图并检索那一派，本文的主要对照立场
- [DuckDB](#) — 基准数据层用的进程内分析型数据库